

Lesson 4: Java Program Structure And Code Comments

Generated Jul 5, 2026 10:56 PM

What We'll Learn

By the end of this lesson, we can:

- Identify class name, main method, statements, comments, and variables.
- Explain the purpose of each Java program part.
- Add comments to describe what code does.

Words We'll Use

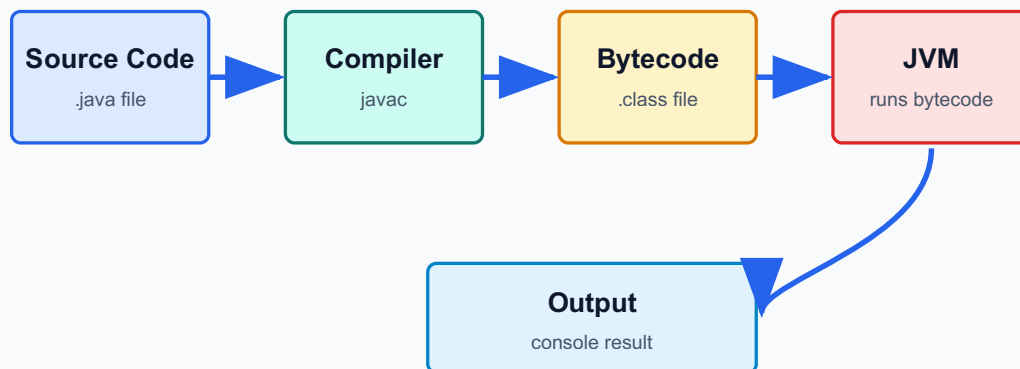
- Comment - note in code ignored by Java.
- Variable - named storage for a value.
- Method - block of code that performs a task.
- Brace - curly symbol used to group code blocks.
- Identifier - name used for classes, variables, or methods.

Before We Start

Which parts of a short Java program look like names, actions, notes, or containers?

Let's Understand

How Java Tools Work Together



Let's work through Lesson 4: Java Program Structure And Code Comments together. Our focus is labeling and explaining the basic structure of a Java program. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- Java is a programming language we use to write instructions for a computer.
- A Java program is written as source code, saved in a `.java` file, compiled, then run.
- The JDK gives us the tools for building Java programs; the JVM runs the compiled bytecode.
- An IDE helps us write, organize, run, and debug code in one workspace.
- A beginner Java program usually has a class, a main method, statements, braces, and comments.
- We read code from the outside container toward the inside instructions, then we trace what will display.

How we study it step by step:

- First, we connect the lesson to a familiar situation: Which parts of a short Java program look like names, actions, notes, or containers?
- Next, we name the important parts so everyone uses the same vocabulary.
- Then, we study Class declaration and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Main method and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Statements and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Comments and ask: what does it do, where do we see it, and how can we check it?
- Then, we study Variables and ask: what does it do, where do we see it, and how can we check it?

- After that, we compare a correct example with a common wrong version.
- We trace the example slowly before running, drawing, or submitting anything.
- Finally, we explain the result in our own words so the activity is not just copying.

Rules and patterns we should remember:

- Rule: Java is case-sensitive, so `System` and `system` are not the same. Example:
`System.out.println("Hi");` works, but `system.out.println("Hi");` causes an error.
 Check: Which word must start with a capital S?
- Rule: Text output must be inside double quotation marks. Example:
`System.out.println("Hello, Java!");` displays Hello, Java! Check: What text will appear on the screen?
- Rule: Many Java statements end with a semicolon. Example: `int score = 90;` is complete, but `int score = 90` is missing its ending mark. Check: What symbol should we add at the end?
- Rule: Braces `{` and `}` must be paired because they show where a block starts and ends.
 Example: `public class Demo { public static void main(String[] args) { } }` has matching class and main braces. Check: How many opening and closing braces can we count?
- Rule: Comments explain code for humans and are ignored when the program runs. Example: `// This line prints a greeting helps us understand the next statement.` Check: Will a comment display in the console?

Common mistakes we will watch for:

- Forgetting capitalization in Java keywords or class names.
- Missing quotation marks, semicolons, or closing braces.
- Thinking comments run as code.
- Changing code without predicting the output first.
- Submitting a screenshot without explaining what happened.

Mini-check before practice:

- What part of the Java program is the container?
- Where does the program start running?
- What line displays output?
- What syntax rule should we check before running?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Label The Program Type: individual Time: 10 minutes Instruction: Identify the main parts of a Java program. Student task:

- Label the class name.
- Label the main method.
- Label two variables.
- Label one output statement.
- Label one comment.

Code / model:

```
public class StudentInfo {  
    public static void main(String[] args) {  
        String name = "Ana";  
        int age = 18;  
        double grade = 95.5;  
        System.out.println(name + " is " + age + " years old.");  
        System.out.println("Grade: " + grade);  
    }  
}
```

Output: Submit a labeled copy or a table of line number and program part.

Activity 2: Brace Pair Check Type: pair Time: 7 minutes Instruction: Trace which braces belong together. Student task:

- Count opening braces.
- Count closing braces.
- Draw pairs or write Class block and Main block.
- Explain what happens if one brace is missing.

Output: Submit brace count and one explanation.

Activity 3: Comment The Code Type: hands-on coding Time: 10 minutes Instruction: Add helpful comments that explain code purpose. Student task:

- Add one comment before the variable declarations.
- Add one comment before output statements.
- Do not comment every single word.
- Run the code to confirm comments do not display.

Output: Submit code with two meaningful comments and output.

Activity 4: Variable Purpose Table Type: individual Time: 8 minutes Instruction: Explain what each variable stores. Student task:

- Create columns: variable, data type, value, purpose.
- Fill in name, age, and grade.
- Write one sentence explaining why variables are useful.

Output: Submit the table and sentence.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Check program-part vocabulary. Student task:

- What part starts the Java program?
- What part stores a value?
- What part is ignored by Java but helps humans?

Output: Submit three answers.

Now We Apply

Now we apply it through these tasks:

1. Write a Java program with at least two variables and two comments.
2. Create a table: Java Part and Purpose.
3. Exchange code and label each other's program.

Challenge Task

Submit an annotated Java program with class, main method, three variables, three output statements, and comments explaining each section.

How Our Work Will Be Checked

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning:

- Labels correctly identify class, main, variables, comments, statements.
- Comments explain purpose, not just repeat code.
- Annotated program is readable.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

Student Activities

Java Program Walk-Through: label class, main, statements, comments, variables. Color Code the Program. Comment the Code: add comments to explain three lines. Parts Matching: match Java part to its purpose. Explain to a Partner: describe what a variable stores. Code Labeling Quiz. Write a Java program with at least two variables and two comments. Create a table: Java Part and Purpose. Exchange code and label each other's program.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.